JAVA DSA • INTERVIEW PREPARATION

TIME • SPACE • AUXILIARY COMPLEXITY ANALYSIS ULTIMATE INTERVIEW HANDBOOK

Big-O • Big-Theta • Big-Omega • Loops • Recursion • Arrays • Strings • Hashing • Sorting • Trees • Graphs • Java Memory

Is handbook ka goal: code dekhkar sirf answer yaad nahi karna, balki logically prove karna seekhna hai ki algorithm kitna time leta hai, kitni memory use karta hai, aur auxiliary space aur total space mein kya difference hai.

Language: Simple Hinglish explanations + complete Java examples + dry runs + interview-style reasoning.

Level: Beginner se advanced interview preparation tak.

Read • Trace • Count • Simplify • Explain

How to use this handbook

Har example ke liye ye 4 steps follow karo: (1) input size define karo, (2) basic operation count karo, (3) dominant term identify karo, (4) auxiliary space ko output space se alag likho. Interview mein final answer ke saath reasoning bhi explain karo.

Pass	Action	Expected skill
1 — Understand	Definitions, growth rates aur examples padho.	Terms ka exact meaning clear ho.
2 — Count	Loops, nested loops, recursion calls manually count karo.	Code se complexity derive kar pao.
3 — Implement	Java examples khud type aur run karo.	Syntax aur behavior samajh aaye.
4 — Explain	Har solution ka time, auxiliary space aur output space bolo.	Interview-ready explanation.
5 — Compare	Brute force aur optimized versions compare karo.	Trade-offs identify kar pao.

Contents

- 1. Complexity ka foundation: input size and operations
- 2. Big-O, Big-Omega, Big-Theta and asymptotic thinking
- 3. Growth-rate ranking and common complexity classes
- 4. Time complexity: counting statements and loops
- 5. Nested loops, dependent loops and logarithmic loops
- 6. Best, average and worst-case analysis
- 7. Space complexity: total space vs auxiliary space
- 8. Recursion space and recurrence intuition
- 9. Java data structures: actual operation costs
- 10. Arrays, strings, hashing, sorting and searching
- 11. Two pointers, sliding window, stack, queue and heap
- 12. Trees, BST, graph traversal and graph representations
- 13. Amortized analysis: dynamic arrays and HashMap intuition
- 14. Java memory: stack, heap, references and object overhead
- 15. Interview traps, worked examples and practice set
- 16. One-page revision sheet and answer templates

1. Complexity ka foundation

Algorithm complexity input size badhne par resources ke use ko describe karti hai. DSA interviews mein usually do resources discuss hote hain: time (kitne operations) aur space (kitni memory).

Input size N kya hota hai?

- Array ke liye N = array.length.
- String ke liye N = string.length() (Java String UTF-16 code units use karti hai; Unicode code points ka count alag ho sakta hai).
- Matrix ke liye R rows aur C columns; total cells R × C.
- Graph ke liye V = vertices aur E = edges.
- Integer input ke liye kabhi N = numeric value, kabhi number of bits/digits. Question ke context se define karo.

Golden rule: Complexity bolne se pehle input size define karo. “O(n)” tabhi meaningful hai jab n ka meaning clear ho.

Operation count ka idea

Exact CPU time machine, language, compiler, memory aur input par depend karta hai. Big-O exact seconds nahi batata. Ye growth pattern batata hai.

Example: linear scan

```
static boolean contains(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return true;
    }
    return false;
}
```

Best case: first element match ho to O(1). Worst case: last element match ho ya target absent ho to O(n). Auxiliary space: O(1), kyunki fixed number of variables use hote hain.

Primitive operation kaise count karein?

- Assignment, comparison, arithmetic, array access, increment ko basic constant-time operations maan sakte hain for standard interview analysis.
- Loop ke andar O(1) work n times chale toh O(n).
- Agar har iteration mein O(n) work hai aur n iterations hain toh $O(n^2)$.
- Exact count jaise 3n + 7 ko asymptotically O(n) likhte hain.

2. Big-O, Big-Omega and Big-Theta

Big-O — upper bound

Big-O batata hai ki sufficiently large input ke liye function ek upper growth bound ke andar rahega. Interview shorthand mein ise aksar worst-case complexity ke liye use kiya jata hai, lekin mathematically Big-O khud upper bound hai; worst-case aur Big-O same concept nahi hain.

Big-Omega — lower bound

Big-Omega lower growth bound batata hai. Best-case analysis mein aksar use hota hai, lekin mathematically Omega bhi kisi specified function/analysis ka lower bound hai.

Big-Theta — tight bound

Theta tab jab upper aur lower dono bounds same growth class mein hon. Example: $4n^2 + 3n + 9 = \Theta(n^2)$.

Notation	Meaning	Common interview interpretation	Example
O(f(n))	Asymptotic upper bound	Often worst-case upper growth	3n+5 = O(n)
Ω(f(n))	Asymptotic lower bound	Often best-case lower growth	3n+5 = Ω(n)
Θ(f(n))	Tight asymptotic bound	Exact growth class up to constants	3n+5 = Θ(n)

Important distinction: Big-O is not automatically “worst case”, Omega is not automatically “best case”. First specify the case/function, then state its asymptotic bound.

Constants and lower-order terms

Suppose $T(n) = 7n^2 + 20n + 100$. Large n par n^2 term dominate karega. Constant 7 aur lower terms ignore hote hain. Therefore $T(n) = \Theta(n^2)$.

Expression	Simplified growth	Reason
9n + 50	Θ(n)	Linear term dominates.
$4n^2 + 8n + 2$	$\Theta(n^2)$	Quadratic term dominates.
$n^3 + 100n^2$	$\Theta(n^3)$	Cubic term dominates.
$5 \log_2 n + 30$	Θ(log n)	Logarithm base is a constant factor.
n log n + 3n	Θ(n log n)	n log n grows faster than n.

Log base kyun ignore hota hai?

Change-of-base formula: log_a(n) = log_b(n) / log_b(a). Agar a aur b constants hain, denominator constant hai. Isliye asymptotic class Θ(log n) same rehti hai.

3. Growth-rate ranking

Generally, same hardware par input bahut bada hone par slower-growing complexity zyada scalable hoti hai.

Growth	Name	Typical example	Scalability
O(1)	Constant	Array index access	Excellent
O(log n)	Logarithmic	Binary search	Excellent
O(√n)	Square root	Trial division optimization	Very good
O(n)	Linear	Single traversal	Good
O(n log n)	Linearithmic	Merge sort, heap sort	Good
$O(n^2)$	Quadratic	All pairs / simple nested loops	Limited
$O(n^3)$	Cubic	Three nested full loops	Poor for large n
$O(2^n)$	Exponential	All subsets / naive Fibonacci	Very poor
O(n!)	Factorial	All permutations	Extremely expensive

Approximate operation counts at n = 1,000

Complexity	Approximate count	Practical interpretation
1	1	Constant work
$\log_2 n$	10	Very small
n	1,000	Usually manageable
$n \log_2 n$	~9,966	Often manageable
n^2	1,000,000	May be manageable depending on constants
n^3	1,000,000,000	Usually too much
2^n	$\sim 10^{301}$	Impossible in practice

Ye counts rough intuition hain, runtime guarantee nahi. Actual feasibility time limit, language, constant factors, memory and input constraints par depend karti hai.

Complexity comparison visual intuition

If n doubles: O(1) unchanged; O(log n) increases slightly; O(n) doubles; O(n log n) a little more than doubles; $O(n^2)$ becomes four times; $O(n^3)$ becomes eight times; $O(2^n)$ squares.

4. Time complexity — statements and loops

O(1): constant work

Constant-time example

```
static int calculate(int x) {
    int a = x + 10;
    int b = a * 2;
    return b;
}
```

Input x kitna bhi bada ho, fixed number of operations execute hote hain. Time Θ(1), auxiliary space Θ(1).

O(n): single loop

Linear loop

```
static int sum(int[] a) {
    int total = 0;
    for (int i = 0; i < a.length; i++) {
        total += a[i];
    }
    return total;
}
```

Loop n baar execute hota hai; each iteration O(1). Total Θ(n). Auxiliary space Θ(1).

O(n): two separate loops add hote hain

Two sequential loops

```
for (int i = 0; i < n; i++) {
    doConstantWork();
}
for (int j = 0; j < n; j++) {
    doConstantWork();
}
```

Total n + n = 2n operations. Asymptotically Θ(n), not O(n²). Sequential loops multiply nahi, add hote hain.

O(n²): nested loops

Nested full loops

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        doConstantWork();
    }
}
```

Outer n times, inner har outer iteration mein n times. Total n × n = n², so Θ(n²).

Triangular nested loop

Inner loop depends on outer index

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j <= i; j++) {
        doConstantWork();
    }
}
```

Total iterations = 1 + 2 + ... + n = n(n+1)/2 = Θ(n²). Inner loop har baar n times nahi chalta, lekin final growth quadratic hi hai.

5. Dependent loops, logarithms and tricky patterns

Loop variable double ho raha hai: O(log n)

Doubling loop

```
for (int i = 1; i < n; i *= 2) {
    doConstantWork();
}
```

i values: 1, 2, 4, 8, 16, ... k iterations ke baad i = 2^k. Stop jab 2^k ≥ n, so k ≈ $\log_2 n$. Time Θ(log n), auxiliary space Θ(1).

Loop variable half ho raha hai: O(log n)

Halving loop

```
for (int i = n; i > 1; i /= 2) {
    doConstantWork();
}
```

n, n/2, n/4, ... Number of halvings $\log_2 n$ ke proportional hai.

Nested logarithmic loop

Two logarithmic loops

```
for (int i = 1; i < n; i *= 2) {
    for (int j = 1; j < n; j *= 2) {
        doConstantWork();
    }
}
```

Outer Θ(log n), inner Θ(log n) per outer iteration. Total $\Theta((\log n)^2)$.

O(n log n): every item triggers a log loop

Linear outer + logarithmic inner

```
for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j *= 2) {
        doConstantWork();
    }
}
```

Outer n times and inner log n times, total Θ(n log n).

Not every nested loop is $O(n^2)$

Two pointers move forward together

```
int left = 0;
int right = n - 1;

while (left < right) {
    if (condition()) {
        left++;
    } else {
        right--;
    }
}
```

Although two indices are involved, total pointer moves at most n. Time O(n), not $O(n^2)$. Count total iterations across the whole algorithm, not just loop nesting.

Loop that shrinks by a constant

If i decreases by 3 each time, iterations ≈ n/3 = Θ(n). Constant step size gives linear complexity. If i is multiplied/divided by a constant factor, complexity is logarithmic.

6. Best, average and worst case

Same algorithm ka runtime input arrangement par depend kar sakta hai. Teen common cases ko alag samjho.

Case	Meaning	Linear search example
Best	Most favorable valid input	Target first index: Θ(1)
Average	Expected/typical cost under a stated input distribution	Uniform target position: Θ(n)
Worst	Most expensive input of given size	Target last/absent: Θ(n)

Linear search: full reasoning

Search

```
static int search(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return i;
    }
    return -1;
}
```

Best case mein first comparison successful: 1 comparison = Θ(1). Worst case mein n comparisons: Θ(n). Average successful search with target equally likely at each index requires (n+1)/2 comparisons, which is Θ(n).

Binary search: case distinction

Sorted array mein middle compare hota hai; har step search space half hota hai. Worst-case Θ(log n). Best case middle element target ho toh Θ(1). Iterative version auxiliary space Θ(1); recursive version call-stack space Θ(log n).

When to report which?

- Agar question explicitly worst-case poochta hai, worst-case do.
- Agar standard DSA solution discuss kar rahe ho, usually worst-case time and auxiliary space report karo, unless average/amortized behavior important ho.
- Average-case claim karte waqt assumptions mention karo; input distribution specify kiye bina average vague ho sakta hai.

7. Space complexity: total space vs auxiliary space

Total space

Algorithm execute karte waqt use hone wali total memory ka broad measure. Ismein input storage, output storage, auxiliary data structures, recursion stack, temporary objects sab include ho sakte hain—analysis convention clearly state karo.

Auxiliary space

Input ko store karne wali memory ko exclude karke algorithm ki extra working memory. Interviews mein jab “space complexity” poochha jata hai, aksar auxiliary space intended hoti hai, lekin hamesha clarify karo.

Most important distinction: Auxiliary space = extra working memory beyond input. Output array/list ko usually auxiliary space se separate report karte hain. Total extra memory including output ko bhi mention kar sakte ho.

Example 1: in-place array sum

O(1) auxiliary space

```
static long sum(int[] nums) {
    long total = 0;
    for (int x : nums) {
        total += x;
    }
    return total;
}
```

Input array n elements already exist. Algorithm creates only a fixed number of variables. Auxiliary space Θ(1). If input storage is included, total memory includes Θ(n) input.

Example 2: output array

Output array is linear

```
static int[] copyAndDouble(int[] nums) {
    int[] result = new int[nums.length];

    for (int i = 0; i < nums.length; i++) {
        result[i] = nums[i] * 2;
    }
    return result;
}
```

Output space Θ(n). Auxiliary working space excluding output Θ(1). If convention counts all additional allocated memory, extra space Θ(n). State both to avoid ambiguity.

Example 3: HashSet

Set stores up to n distinct values

```
static boolean hasDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();

    for (int x : nums) {
        if (!seen.add(x)) return true;
    }
    return false;
}
```

Worst-case stored set entries O(n), so auxiliary space O(n). Expected time O(n) under normal HashSet hashing assumptions; pathological collision behavior can be worse.

Example 4: matrix allocation

Creating `int[n][m]` allocates Θ(nm) cell storage plus Java array-object overhead. If it is an output matrix, report output space Θ(nm); if temporary working storage, auxiliary space Θ(nm).

8. Recursion: time and call-stack space

Recursive algorithm mein do cheezein count karo: total calls/work (time) and maximum number of simultaneously active calls (recursion stack space). Total calls aur maximum recursion depth same nahi hote.

Linear recursion

Factorial

```
static long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Calls: n, n-1, ..., 1 → Θ(n) calls. Har call O(1) work. Time Θ(n). Maximum active stack depth n → auxiliary stack space Θ(n).

Binary recursion tree: naive Fibonacci

Naive Fibonacci

```
static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Calls exponential number mein grow karte hain: time O(2^n) (more tightly Θ(φ^n) for standard recurrence). Maximum recursion depth O(n), so auxiliary stack space O(n), not O(2^n).

Binary search recursion

Recursive binary search

```
static int binarySearch(int[] a, int left, int right, int target) {
    if (left > right) return -1;

    int mid = left + (right - left) / 2;
    if (a[mid] == target) return mid;

    if (a[mid] < target) {
        return binarySearch(a, mid + 1, right, target);
    }
    return binarySearch(a, left, mid - 1, target);
}
```

Each call halves range: Θ(log n) calls and Θ(log n) call-stack space. Time Θ(log n).

Recursion space formula

> Recursion auxiliary space is usually proportional to maximum recursion depth × memory retained per active call. If each frame holds O(1) local data and depth is d, stack space O(d).

9. Java data structures — operation complexity

Interview mein data structure ki complexity implementation aur assumptions par depend karti hai. Neeche common Java collections ki typical expected/amortized behavior di gayi hai.

Java structure	Operation	Typical time	Extra space intuition
Array / int[]	index get/set	O(1)	Array storage O(n)
ArrayList	get(i)	O(1)	Backing array O(n)
ArrayList	append	Amortized O(1)	Occasional resize
ArrayList	insert/remove middle	O(n)	Shift elements
LinkedList	add/remove ends	O(1)	Node per element
LinkedList	get(i)	O(n)	Node links
ArrayDeque	add/remove ends	Amortized O(1)	Resizable circular array
HashMap / HashSet	get/put/contains	Expected O(1)	O(n) entries
TreeMap / TreeSet	search/insert/delete	O(log n)	O(n) nodes
PriorityQueue	peek	O(1)	Heap O(n)
PriorityQueue	offer/poll	O(log n)	Heap O(n)

Java API complexity can depend on implementation details and JDK version. For interviews, explain the standard expected/amortized model and avoid claiming every operation is worst-case O(1).

HashMap nuance

- Average/expected get and put are O(1) with well-distributed hashes.
- Worst-case behavior can degrade under heavy collisions; modern Java has collision-handling optimizations for suitable bucket conditions, but don't casually call it guaranteed worst-case O(1).
- HashMap uses extra memory proportional to number of entries; capacity and load factor affect actual allocation.

ArrayList append: why amortized O(1)?

Most appends place an element in the current backing array in O(1). Occasionally capacity is insufficient, so a larger array is allocated and elements are copied in O(n). Across many appends, total copying is linear, making average cost per append amortized O(1).

10. Complexity of common algorithms

Algorithm	Time	Auxiliary space	Notes
Linear search	Best O(1), worst O(n)	O(1)	Unsorted array
Binary search	Best O(1), worst O(log n)	Iterative O(1); recursive O(log n)	Sorted random-access data
Bubble sort	Best O(n) optimized; worst $O(n^2)$	O(1)	Stable in standard version
Selection sort	$\Theta(n^2)$	O(1)	Few swaps
Insertion sort	Best O(n); worst $O(n^2)$	O(1)	Good for small/nearly sorted data
Merge sort	$\Theta(n \log n)$	O(n) for arrays	Stable; standard merge buffer
Quick sort	Average $O(n \log n)$; worst $O(n^2)$	Expected O(log n) stack; worst O(n)	Partition-dependent
Heap sort	$\Theta(n \log n)$	O(1) auxiliary for in-place array heap sort	Not stable
Counting sort	O(n + k)	O(n + k)	Key range k
Radix sort	O(d(n + b))	O(n + b)	d digits, base b

Why merge sort uses O(n) extra space?

Array merge sort typically creates temporary arrays/buffer to merge halves. Recursion depth is O(log n), but the dominant extra buffer is O(n); total auxiliary space remains O(n).

Why quicksort stack space is different from time?

Partitioning takes O(n) per level. Balanced recursion has O(log n) levels; highly unbalanced recursion can have O(n) depth. Thus expected stack O(log n), worst O(n), while worst-case time $O(n^2)$.

11. Complexity of common DSA patterns

Pattern / algorithm	Time	Auxiliary space	Key condition
Two pointers on sorted array	O(n)	O(1)	Pointers move monotonically
Fixed sliding window	O(n)	O(1) state; output may O(n)	Add/remove in O(1)
Variable sliding window	Often O(n)	O(1) or O(k) for map/set	Each pointer moves at most n
Monotonic stack	O(n)	O(n)	Each index pushed/popped once
Monotonic deque	O(n)	O(k) or O(n)	Each index enters/leaves once
BFS	O(V+E)	O(V)	Adjacency list
DFS	O(V+E)	O(V)	Visited + recursion/explicit stack
Dijkstra with binary heap	O((V+E) log V) typical	O(V+E)	Non-negative edge weights
Topological sort	O(V+E)	O(V)	DAG
Heap build from array	O(n)	O(1) auxiliary in-place	Bottom-up heapify
Heap insertion/removal	O(log n)	O(1) per operation	Binary heap

Why sliding window can be O(n) despite a while loop?

Outer right pointer n times move karta hai. Inner while left pointer ko aage badhata hai, lekin left kabhi peeche nahi jaata aur total maximum n moves karta hai. Dono pointers combined O(2n) = O(n).

Why monotonic stack is O(n)?

Nested while loop dekhkar $O(n^2)$ assume mat karo. Har element stack mein maximum once push aur once pop hota hai. Total stack operations $\leq 2n$, so O(n).

12. Trees and graphs complexity

Binary tree traversal

Operation	Time	Auxiliary space
Preorder / Inorder / Postorder	O(n)	O(h) recursive stack
Level order BFS	O(n)	O(w) queue; worst O(n)
Height calculation	O(n)	O(h) recursive stack
Search in arbitrary binary tree	O(n)	O(h) recursive stack

n = number of nodes, h = tree height, w = maximum width. Balanced tree height O(log n), skewed tree height O(n).

BST operations

BST shape	Search / insert / delete	Height
Balanced	O(log n)	O(log n)
Skewed	O(n)	O(n)
Self-balancing (AVL / Red-Black)	O(log n)	O(log n)

Graph representation matters

Representation	Space	Check if edge u-v exists
Adjacency matrix	$O(V^2)$	O(1)
Adjacency list	O(V+E)	O(degree(u)) typical list scan
Edge list	O(E)	O(E) scan

BFS with adjacency list

Java BFS

```
static void bfs(List<List<Integer>> graph, int start) {
    int n = graph.size();
    boolean[] visited = new boolean[n];
    Queue<Integer> queue = new ArrayDeque<>();

    visited[start] = true;
    queue.offer(start);

    while (!queue.isEmpty()) {
        int node = queue.poll();

        for (int next : graph.get(node)) {
            if (!visited[next]) {
                visited[next] = true;
                queue.offer(next);
            }
        }
    }
}
```

Every vertex is enqueued at most once; every adjacency-list edge is examined a constant number of times (for directed graphs once from its source). Time O(V+E), auxiliary space O(V) for visited and queue, excluding graph input.

13. Amortized analysis

Amortized analysis multiple operations ke sequence ka average cost bound karta hai. Ye probabilistic average-case nahi hai; it guarantees a bound over a sequence of operations.

Dynamic array append

ArrayList jaise dynamic array mein capacity full hone par larger backing array create hota hai and old elements copy hote hain. Single resize append O(n) ho sakta hai, but n appends ka total work geometric growth ke under O(n), so amortized append O(1).

Operation	Single-operation cost	Amortized cost
Normal append with free capacity	O(1)	O(1)
Append that triggers resize	O(n)	O(1) over sequence
n appends total	O(n) total copying + writes	O(n) total

Three analysis ideas

- Aggregate method: total cost of n operations bound karo, then divide by n.
- Accounting method: cheap operations se credits save karke expensive future resize pay karne ka intuition.
- Potential method: data structure ki stored potential/credit change ko use karke amortized cost derive karo.

Interview phrase: “An individual append can be O(n) during resize, but append is amortized O(1) over a sequence of operations.”

14. Java memory: stack, heap and references

Java program memory ko interview-level par samajhne ke liye local variables, object allocation aur references ka difference clear rakho. JVM implementation details complex hain; ye conceptual model hai.

Local reference vs object

Array allocation

```
static void process() {
    int n = 5;
    int[] data = new int[n];
    data[0] = 10;
}
```

`data` local reference variable hai; array object dynamically allocated hota hai. Conceptually reference local execution frame se associated hota hai, array object heap par hota hai. Exact JVM placement optimizations ke saath differ kar sakti hai.

Space analysis example

Recursive array processing

```
static int sum(int[] a, int index) {
    if (index == a.length) return 0;
    return a[index] + sum(a, index + 1);
}
```

Input array O(n) storage already occupies karta hai. Recursive calls O(n) deep hain, so auxiliary call-stack space O(n). Algorithm n elements process karta hai, so time O(n).

Object memory overhead

- Java objects/arrays have headers and alignment overhead; exact bytes JVM, compressed references and runtime configuration par depend karte hain.
- Interview Big-O analysis mein generally each stored element/object ko constant-size unit maana jata hai.
- HashMap entries, linked-list nodes and boxed primitives can use more memory than raw primitive arrays.
- `int[]` generally memory-efficient hota hai compared with `ArrayList` because Integer boxing/reference/object overhead may exist.

Recursion stack vs heap

Recursion calls ke active frames stack-like execution memory use karte hain; `new` se allocated objects generally heap par hote hain. Complexity answer mein recursion depth ko auxiliary space ke roop mein count karna na bhoolo.

15. Worked interview examples

Example A — Find maximum in array

Java

```
static int findMax(int[] a) {
    int max = a[0];
    for (int i = 1; i < a.length; i++) {
        if (a[i] > max) max = a[i];
    }
    return max;
}
```

Time: Θ(n), because n-1 comparisons. Auxiliary space: Θ(1). Input: Θ(n) already provided. Empty array handling should be defined separately.

Example B — Create reversed copy

Java

```
static int[] reversedCopy(int[] a) {
    int[] result = new int[a.length];
    for (int i = 0; i < a.length; i++) {
        result[i] = a[a.length - 1 - i];
    }
    return result;
}
```

Time: Θ(n). Output space: Θ(n). Auxiliary space excluding output: Θ(1). Additional allocated space including output: Θ(n).

Example C — Reverse in place

Java

```
static void reverseInPlace(int[] a) {
    int left = 0;
    int right = a.length - 1;

    while (left < right) {
        int temp = a[left];
        a[left] = a[right];
        a[right] = temp;
        left++;
        right--;
    }
}
```

Time: Θ(n). Auxiliary space: Θ(1). Input array itself is modified; no second array.

Example D — All pairs

Java

```
static void printPairs(int[] a) {
    for (int i = 0; i < a.length; i++) {
        for (int j = i + 1; j < a.length; j++) {
            System.out.println(a[i] + ", " + a[j]);
        }
    }
}
```

Number of pairs = n(n-1)/2, so time Θ(n²). Auxiliary space O(1) if output is streamed/printed and not stored. If all pairs are collected into a list, output space Θ(n²).

16. Interview traps and common mistakes

- Two sequential loops are usually O(n), not O(n²).
- Nested loops are not automatically O(n²); count total iterations and dependencies.
- A while loop inside a for loop can still be O(n) when a pointer moves only forward overall.
- Sorting an array first usually adds O(n log n) time even if the next scan is O(n).
- Returning an output array/list requires output-space accounting.
- Input space and auxiliary space are not interchangeable.
- Recursive calls consume call-stack space even if no array or collection is created.
- HashMap/HashSet O(1) is expected/average behavior, not a universal worst-case guarantee.
- ArrayList append is amortized O(1), while insert/remove at an arbitrary middle index is O(n).
- Binary search needs sorted data and a suitable access model; linked-list middle access is not O(1).
- An O(n²) algorithm may be fine for small n; constraints determine practical feasibility.
- Big-O drops constants and lower terms; it does not predict exact runtime seconds.
- Best, average and worst cases are input-case distinctions; O, Ω and Θ are bound notations.
- Two recursive calls do not automatically mean O(2^n); derive the recurrence/call tree.
- Space complexity should state whether output and input are included.

How to derive a loop quickly

Code shape	Typical time	Reason
for i = 0 to n-1	O(n)	n iterations
for i = 0 to n-1; for j = 0 to n-1	O(n²)	n × n
for i = 1; i < n; i *= 2	O(log n)	Multiplicative growth
for i = n; i > 0; i /= 2	O(log n)	Repeated halving
n outer × log n inner	O(n log n)	Multiply iteration counts
two pointers, each moves ≤ n	O(n)	Total moves are linear

17. Practice set — calculate the complexity

Pehle khud answer nikalo. Har question ke baad explanation check karo. n input size hai.

Q1. Single loop

Q1. Single loop

```
for (int i=0; i<n; i++) work();
```

Answer: Time Θ(n); auxiliary space Θ(1).

Q2. Two sequential loops

```
for (int i=0;i<n;i++) work(); for (int j=0;j<n;j++) work();
```

Answer: Θ(n+n) = Θ(n); auxiliary space Θ(1).

Q3. Nested loops

```
for (int i=0;i<n;i++) for (int j=0;j<n;j++) work();
```

Answer: Θ(n²); auxiliary space Θ(1).

Q4. Doubling

```
for (int i=1;i<n;i*=2) work();
```

Answer: Θ(log n); auxiliary space Θ(1).

Q5. Outer n, inner log n

```
for (int i=0;i<n;i++) for (int j=1;j<n;j*=2) work();
```

Answer: Θ(n log n); auxiliary space Θ(1).

Q6. Allocate array

```
int[] b = new int[n];
```

Answer: Time Θ(n) to initialize Java array to default values; auxiliary/output allocation Θ(n), depending on role.

Q7. Recursive countdown

```
f(n) calls f(n-1) once until base case.
```

Answer: Time Θ(n); recursion stack Θ(n).

Q8. Binary search

```
Search sorted array by halving range.
```

Answer: Worst time Θ(log n); iterative auxiliary Θ(1), recursive stack Θ(log n).

Q9. HashSet duplicate check

```
Insert each item; stop if already present.
```

Answer: Expected time O(n); auxiliary space O(n).

Q10. Monotonic stack

```
Each index pushed once and popped at most once.
```

Q1. Single loop

```
for (int i=0; i<n; i++) work();
```

Answer: Time Θ(n); auxiliary space Θ(1).

Q2. Two sequential loops

```
for (int i=0;i<n;i++) work(); for (int j=0;j<n;j++) work();
```

Answer: Θ(n+n) = Θ(n); auxiliary space Θ(1).

Q3. Nested loops

```
for (int i=0;i<n;i++) for (int j=0;j<n;j++) work();
```

Answer: Θ(n²); auxiliary space Θ(1).

Q4. Doubling

```
for (int i=1;i<n;i*=2) work();
```

Answer: Θ(log n); auxiliary space Θ(1).

Q5. Outer n, inner log n

```
for (int i=0;i<n;i++) for (int j=1;j<n;j*=2) work();
```

Answer: Θ(n log n); auxiliary space Θ(1).

Q6. Allocate array

```
int[] b = new int[n];
```

Answer: Time Θ(n) to initialize Java array to default values; auxiliary/output allocation Θ(n), depending on role.

Q7. Recursive countdown

```
f(n) calls f(n-1) once until base case.
```

Answer: Time Θ(n); recursion stack Θ(n).

Q8. Binary search

```
Search sorted array by halving range.
```

Answer: Worst time Θ(log n); iterative auxiliary Θ(1), recursive stack Θ(log n).

Q9. HashSet duplicate check

```
Insert each item; stop if already present.
```

Answer: Expected time O(n); auxiliary space O(n).

Q10. Monotonic stack

```
Each index pushed once and popped at most once.
```

Answer: Time O(n); auxiliary space O(n).